

Course Name:	Artificial Intelligence Laboratory	Semester:	VI
Date of Performance:	_10_ / _02_ / _26_	DIV/ Batch No:	C-3
Student Name:	Shreyans Tatiya	Roll No:	16010123325

Experiment No:3

Title: Implementation of uninformed search algorithms

Aim: Implementation of uninformed search algorithms – BFS,DFS, DLS for the given problem

Objectives of the Experiment:

By the end of this experiment, students will be able to:

1. Examine the efficiency and performance of uninformed search algorithms in solving various problems.

COs to be achieved:

CO2: Analyse and solve problems for goal based agent architecture (searching and planning algorithms)

Books/ Journals/ Websites referred:

1. https://www.ccs.neu.edu/home/rplatt/cs5100_spring2017/slides/graph_search.pdf
2. <https://www.cs.usfca.edu/~galles/cs662/lecture/lecture5.pdf>

Theory:

Problem-Solving Agent

A problem-solving agent is designed to find solutions to well-defined problems. This agent typically follows these steps:

- **Formulate the Problem:** Define the initial state, goal state, and possible actions.
- **Search for a Solution:** Use an appropriate search strategy to explore the problem space.
- **Execute the Solution:** Apply the sequence of actions derived from the search.

Uninformed Search Algorithms

Uninformed search algorithms, also known as blind search algorithms, are basic search strategies that explore the search space without any additional information about the goal's location beyond what is provided in the problem definition.

Uninformed Searching Technique

- **Breadth-First Search (BFS):** Explores all nodes at the present depth level before moving on to nodes at the next depth level. Complete and optimal if the cost is uniform.
- **Depth-First Search (DFS):** Explores as far as possible along each branch before backtracking. Not complete or optimal, but requires less memory.
- **Depth-Limited Search (DLS):** Depth-first search with a predetermined depth limit. Can overcome infinite path problems.

Activity :

Problem Formulation: Define initial state, goal state, and actions for the given problem

Construct a **state-space tree** for the given problem.

8-Puzzle Problem

Problem Description

The problem is to solve an **8-Puzzle configuration** where numbered tiles (1–8) are arranged on a 3×3 board with one blank space represented by **0**.

The objective is to reach the goal configuration from a given initial configuration by sliding tiles into the blank space.

(i) Initial State

The initial state represents the starting arrangement of the puzzle tiles.

Example Initial State (as used in the code):

123

405

678

Here:

- Numbers **1–8** represent tiles
- **0** represents the blank space
- Tiles can slide only into the blank position.

This corresponds to:

```
runTest("123405678");
```

(ii) State Space

The state space consists of **all possible configurations of the 8 tiles and one blank space** that can be generated by sliding tiles.

Each state is represented as a **string of 9 digits**:

"123456780"

where:

- Each digit represents a tile position
- 0 indicates the blank tile.

Example states:

103

425

678

123

450

678

Only valid configurations produced by legal tile moves are considered.

Additionally, the program checks solvability using:

`isSolvable(state)`

which counts inversions to determine whether a solution exists.

(iii) Actions

Actions represent sliding a tile into the blank space.

From any state:

- The blank tile (0) can move:
 - Up
 - Down
 - Left
 - Right

This is implemented in:

`getNextStates(String state)`

Example:

If blank is in center:

123

405

678

Possible actions:

- Move tile above down
- Move tile below up
- Move left tile right
- Move right tile left

Thus:

Actions(s) = All valid moves of the blank tile

(iv) Goal Test

The goal test checks whether the current state matches the goal configuration.

Goal State:

123

456

780

This corresponds to:

`static final String GOAL = "123456780";`

Goal condition:

If current state == GOAL

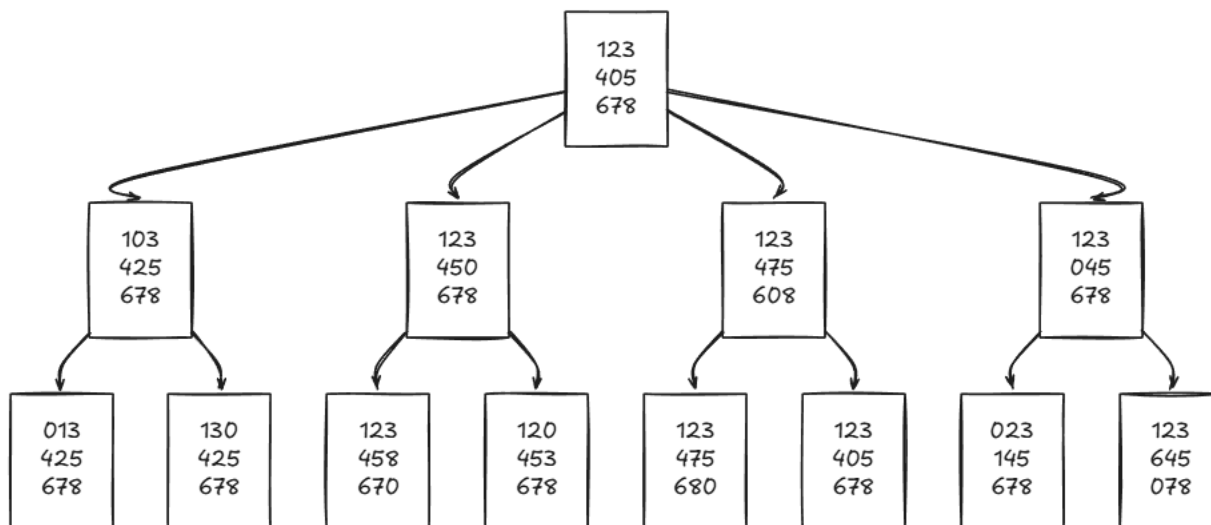
→ Goal achieved

The search algorithms used:

- Breadth-First Search (BFS)
- Iterative Depth-First Search (DFS)
- Depth-Limited Search (DLS)

stop when this goal state is reached.

(v) State Space Tree



Code :

```
import java.util.*;

public class EightPuzzle {

    static final String GOAL =
"123456780";

    // ----- Generate Next
States -----
    static List<String>
getNextStates(String state) {
    List<String> nextStates =
new ArrayList<>();

    int index =
state.indexOf('0');
    int row = index / 3;
    int col = index % 3;

    int[][] moves = {
        {1, 0}, {-1, 0},
{0, 1}, {0, -1}
    };

    for (int[] m : moves) {
        int r = row + m[0];
        int c = col + m[1];

        if (r >= 0 && r < 3 &&
c >= 0 && c < 3) {
            int newIndex = r *
3 + c;

            char[] arr =
state.toCharArray();

            char temp =
arr[index];
            arr[index] =
arr[newIndex];
            arr[newIndex] =
temp;

            nextStates.add(new
String(arr));
        }
    }

    return nextStates;
}
```

```
}

// ----- Solvability Check
-----

static boolean
isSolvable(String state) {
    int inv = 0;
    for (int i = 0; i < 9; i++)
    {
        for (int j = i + 1; j <
9; j++) {
            char a =
state.charAt(i);
            char b =
state.charAt(j);
            if (a != '0' && b
!= '0' && a > b)
                inv++;
        }
    }
    return inv % 2 == 0;
}

// ----- BFS -----
static void bfs(String start) {
    Queue<String> q = new
LinkedList<>();
    Set<String> visited = new
HashSet<>();

    q.add(start);
    visited.add(start);

    while (!q.isEmpty()) {
        String curr = q.poll();

        if (curr.equals(GOAL))
        {
            System.out.println("Goal found
using BFS");
            return;
        }

        for (String next :
getNextStates(curr)) {
            if
(!visited.contains(next)) {
```

```
visited.add(next);

                q.add(next);
            }
        }
    }

    System.out.println("Goal
NOT reachable (BFS)");
}

// ----- ITERATIVE DFS
(FIXED) -----
static void dfsIterative(String
start) {
    Stack<String> stack = new
Stack<>();
    Set<String> visited = new
HashSet<>();

    stack.push(start);

    while (!stack.isEmpty()) {
        String curr =
stack.pop();

        if
(visited.contains(curr))
            continue;

        visited.add(curr);

        if (curr.equals(GOAL))
        {

System.out.println("Goal found
using DFS");

            return;
        }

        for (String next :
getNextStates(curr))
            stack.push(next);
    }

    System.out.println("Goal
NOT reachable using DFS");
}

// ----- Depth Limited
```

Search -----

```
static void dls(String start,
int limit) {
    Stack<Pair> stack = new
Stack<>();
    Set<String> visited = new
HashSet<>();

    stack.push(new Pair(start,
0));

    while (!stack.isEmpty()) {
        Pair p = stack.pop();
        String state = p.state;
        int depth = p.depth;

        if
(visited.contains(state))
            continue;

        visited.add(state);

        if (state.equals(GOAL))
{
System.out.println("Goal found
using DLS");

        return;
        }

        if (depth < limit) {
            for (String next :
getNextStates(state))
                stack.push(new
Pair(next, depth + 1));
        }

        System.out.println("Goal
NOT found within depth limit");
    }

    static class Pair {
        String state;
        int depth;
        Pair(String s, int d) {
            state = s;
            depth = d;
        }
    }
}
```

```
}

// ----- Run Test -----
--
static void runTest(String
start) {
    System.out.println("\nStart
State: " + start);

    if (!isSolvable(start)) {
        System.out.println("Not
solvable.");
        return;
    }

    System.out.println("Running
BFS...");
    bfs(start);

    System.out.println("Running
DFS...");
    dfsIterative(start);

    System.out.println("Running
DLS (limit=15)...");
    dls(start, 15);
}

// ----- MAIN -----
public static void
main(String[] args) {
    runTest("123405678");
    runTest("123456708");
    runTest("125340678");
    runTest("123456870");
}
}
```

Output

```
Start State: 123405678
Running BFS...
Goal found using BFS
Running DFS...
Goal found using DFS
Running DLS (limit=15)...
Goal NOT found within depth limit
```

```
Start State: 123456708
Running BFS...
Goal found using BFS
Running DFS...
Goal found using DFS
Running DLS (limit=15)...
Goal found using DLS
```

```
Start State: 125340678
Running BFS...
Goal found using BFS
Running DFS...
Goal found using DFS
Running DLS (limit=15)...
Goal NOT found within depth limit
```

```
Start State: 123456870
Not solvable.
```

```
=== Code Execution Successful ===
```

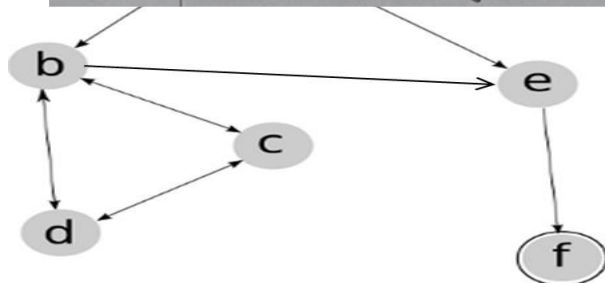
Post Lab Subjective/Objective type Questions:

Question: Answer in brief (Hand written answers)

1. Differentiate between BFS, DFS, DLS and IDS.

Ans:

Algorithm	Full form	Strategy	Data Structure used	Completeness	Optimality
BFS	Breadth First Search	Explores level by level	Queue (FIFO)	Yes	Yes (for eq. step cost)
DFS	Depth first Search	Explores as deep as possible before backtracking	Stack (LIFO)	Not always	No
DLS	Depth Limited Search	DFS with a fixed depth limit	Stack	No	No
IDS	Iterative Deepening Search	Repeating DLS with increasing depth limits	Stack	Yes Yes	Yes (for eq. step cost)



2. Which algorithm guarantees the shortest path? Why?

BFS guarantees the shortest path - when all edges have equal cost
Reason: BFS explores nodes level by level. The first time it reaches the goal node, it has taken the minimum number of edges possible.

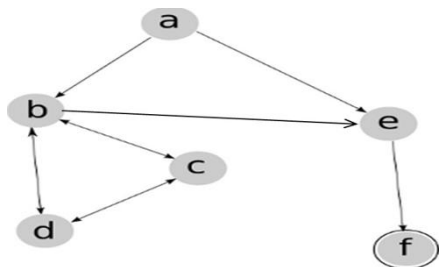
3. What is the disadvantage of DFS?

It does not guarantee the shortest path
It can get stuck in a deep infinite branch or a very deep branch
May waste time exploring long incorrect paths before finding the solution.

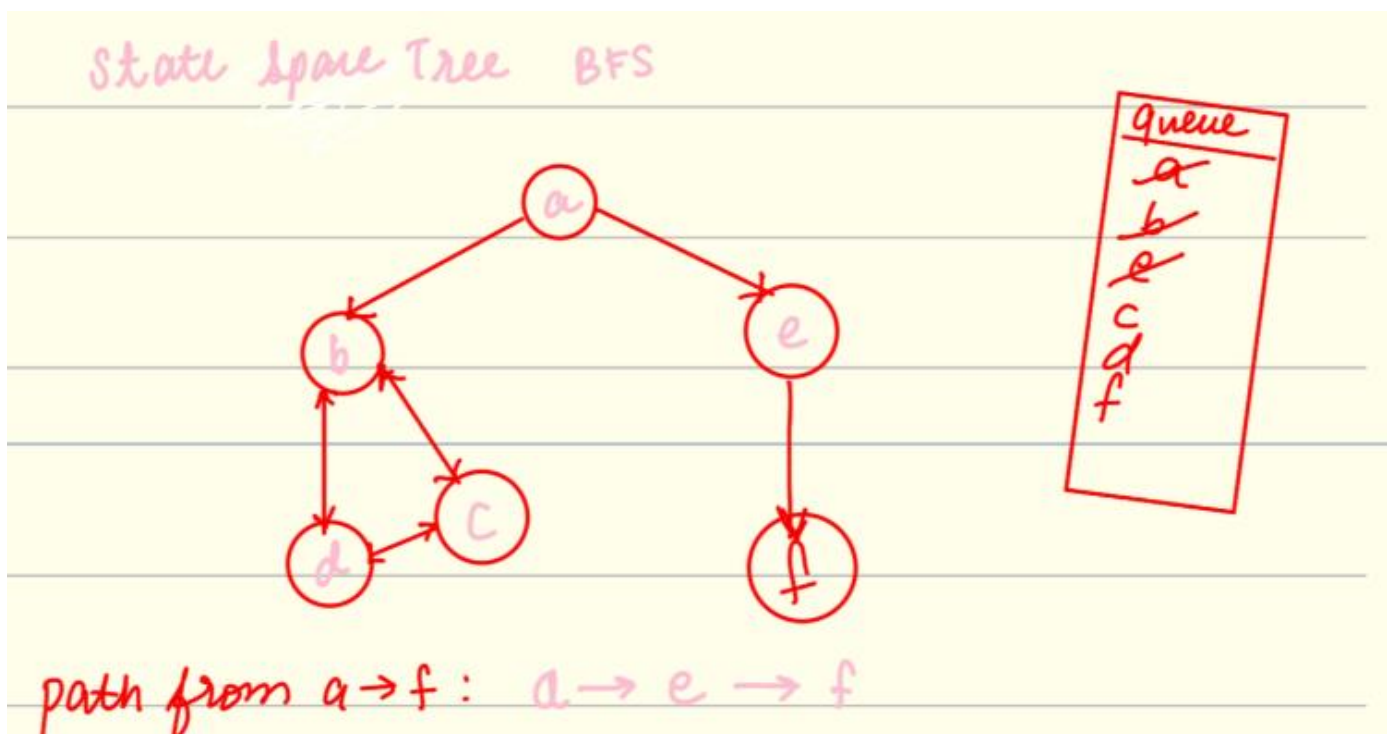
4. Why is DLS preferred over DFS in some cases?

DLS is preferred when -
→ search space is very deep/infinite
→ we want to prevent DFS from going infinitely deep
→ we know the approx depth of the sol.
It controls DFS by restricting max depth.

5. Construct the state space tree. Apply the given algorithm to find the path (from a $\rightarrow$ f).



Ans :





SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)

K. J. Somaiya School of Engineering, Mumbai-77
(Somaiya Vidyavihar University)
Department of Computer Engineering



Conclusion:

The above experiment highlights implementation of the problem – “Map of Romania” using uninformed search algorithms such as BFS, DFS, DLS and highlights the differences in implementation methods of all these algorithms.



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)

K. J. Somaiya School of Engineering, Mumbai-77
(Somaiya Vidyavihar University)
Department of Computer Engineering

